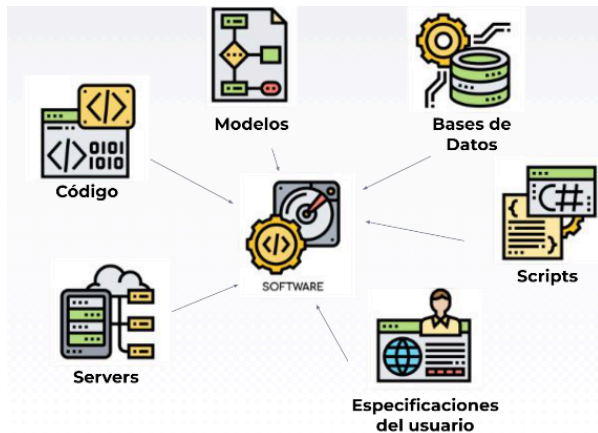


Software Configuration Management (SCM)

La gestión de configuración de SW es una disciplina orientada a administrar la evolución de **productos, procesos y ambientes**, definiendo mecanismos para administrar diferentes versiones de los mismos controlando los cambios efectuados, y auditando y reportando la actividad de cambios realizada.

Su **propósito** es establecer y mantener la integridad de los artefactos del proyecto de SW a lo largo del ciclo de vida del mismo.



Conceptos claves

Ítem de configuración (IC)

- Cualquier elemento involucrado en el desarrollo del producto y que está bajo el control de la gestión de configuración
- Para cada ítem se conoce información específica para poder identificarlo, como por ejemplo nombre, versión, fecha de creación, autos, entre otros atributos



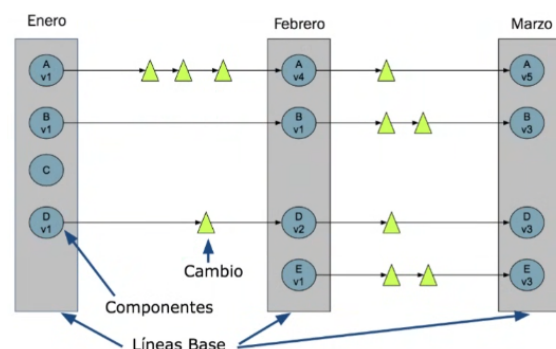
Configuración (de SW)

- Conjunto de ítems de configuración que gestionaremos para la construcción de nuestro producto SW
- La configuración de SW puede o no contener código, dependerá del momento en la construcción del producto SW.
- Ejemplo: Al iniciar un proyecto, la configuración puede estar compuesta únicamente por una especificación de requerimientos de SW



Línea base (Baseline)

- Representa un estado de la configuración de un conjunto de ítems en el ciclo de desarrollo, que puede tomarse como punto de referencia para una siguiente etapa del ciclo
- Se establece porque verifica que esta configuración del ítem o conjunto de ítems satisface (n) algunos requerimientos funcionales o técnicos.



Trazabilidad

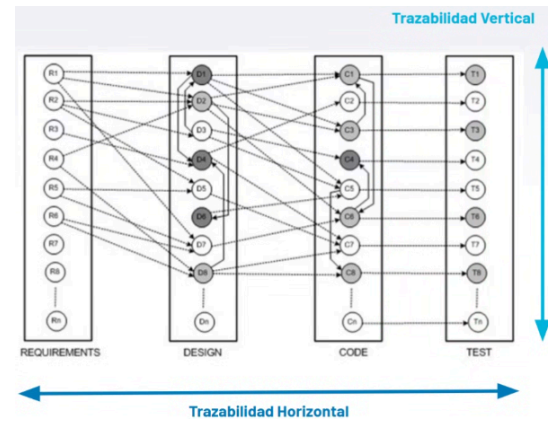
Capacidad de rastrear un ítem de configuración a lo largo de todo el proceso

Horizontal

Representa la capacidad de trazar todos los ítems de configuración generados en cada etapa del proceso por un mismo requerimiento

Vertical

Representa la capacidad de trazar todos los ítems de configuración del mismo tipo, generados en la misma etapa



Branching

- Acción de crear líneas de desarrollo separadas de una línea de trabajo. Se las conoce usualmente como branches
- Estos branches son mecanismos utilizados por los equipos para que puedan desarrollar nuevas funcionalidades utilizando un entorno de trabajo separado del resto.

Ambiente (Environment)

- Un entorno de implementación es una colección de servidores, clusters y servicios configurados que colaboran para proporcionar un entorno para alojar módulos de SW.
- Cada entorno se construye con un propósito, pudiendo tener características funcionalmente similares pero no necesariamente ser iguales a nivel infraestructura.
 - Ejemplo: entorno de preproducción y producción. No necesariamente son iguales a nivel infraestructura pero deberían ser equivalentes a nivel SW.
 - El propósito del ambiente es decidido por la organización o equipo de trabajo. Es decir, cada organización o equipo decide para qué será utilizado el ambiente y en qué etapa del proceso del desarrollo se utilizará.
- Los ambientes comunes en el desarrollo suelen ser desarrollo, staging, producción.

Funciones de la SCM acorde al SWEBOK



Administración del proceso de Gestión de Configuración del SW

Define herramientas, reportes y procesos que serán utilizados, en la organización o el proyecto, para la gestión de configuración

Se encarga de definir cómo serán incorporados los IC 's, cuando se realizaran las líneas base, como será la estrategia de desarrollo en paralelo, y como serán nombradas las versiones, entre otras definiciones.

Generalmente se realiza mediante la definición de un plan de CM o plan de Gestión de Configuración.

Plan de gestión de configuración (CM Plan)

Un plan de SCM define principalmente:

- Lista ítems de configuración (IC), y en qué momento ingresan al sistema de CM
- Define estándares de nombres, jerarquías de directorios, estándares de versionamiento
- Define las estrategias de branching
- Define los procedimientos para crear “builds” y “releases”
- Define reglas de uso de herramientas de CM y el rol de administrador de la configuración.
- Define el contenido de los reportes de auditoría y los momentos en que se ejecutan.

Branching Strategies

- La estrategia de branching es la estrategia que adoptan los equipos de desarrollo de sw al escribir, mergear e implementar código cuando usan un sistema de control de versiones.
- Es esencialmente un conjunto de reglas que los desarrolladores pueden seguir para estipular cómo interactúan con una base de código compartida.
- Tiene como objetivos
 - Mejorar la productividad al garantizar una coordinación adecuada entre los desarrolladores
 - Facilitar el desarrollo paralelo
 - Ayudar a organizar una serie de lanzamientos planificados y estructurados
 - Trazar un camino claro al realizar cambios en el sw hasta la producción
 - Mantenga un código libre de errores donde los desarrolladores puedan corregir problemas rápidamente y hacer que estos cambios vuelvan a la producción sin interrumpir el flujo de trabajo de desarrollo.

- Ejemplos
 - gitflow: Consiste en dos ramas principales que existen a lo largo de todo el ciclo de vida del desarrollo, master y develop y tiene una serie de branches de soporte. En master vive el código productivo
 - github flow
 - Creado por GitHub, busca ser una alternativa simplificada del desarrollo
 - Existe una rama Master de donde parten ramas de desarrollo
 - Cada nueva feature se trabaja en una rama separada que es integrada nuevamente en master cuando el trabajo finaliza
 - trunk based flow: Modelo de branching de control de fuente, donde los desarrolladores colaboran en el código en una sola rama llamada “troncal”, resiste cualquier presión para crear otras ramas de desarrollo de larga duración mediante el empleo de técnicas documentadas. Por lo tanto, evitan fusionar el infierno, no rompen la construcción y viven felices.

Identificación de la configuración

Actividad de definir qué elementos (IC 's) estarán controlados por la gestión de configuración ya que no todos los artefactos necesitan ser gestionados para garantizar la integridad del producto.

La **identificación de configuración** también define momentos o condiciones para establecer una línea de base o bien para liberarla (es decir llevarla a otro ambiente, también conocido como “release”)

Acciones:

- **Identificar código**
 - Buscar fuentes
 - Chequear repositorios
 - Entender la arquitectura
- **Identificar ambientes en ejecución**
 - Tratar de instalar una versión
 - Identificar BD, servers, configuraciones
- **Identificar documentación**
 - Buscar especificaciones
 - Entender cambios anteriores

Elementos a registrar

- Nombre
- Versión
- Autor/Revisores
- Módulos o ítems relacionados
- Última modificación
- Tipo de IC (código, scripts, diagramas, requerimientos, etc)

Control de cambios de la configuración

Asegura que los ítems de configuración mantiene su integridad ante los cambios a través de:

- La **identificación** del propósito del cambio
- La **evaluación del impacto** y aprobación del cambio
- La **planificación** de la incorporación del cambio
- El **control** de la implementación del cambio y su verificación.

El control de cambios de la configuración consiste en establecer un procedimiento de control de cambios y controlar el cambio y la liberación de IC 's a lo largo del ciclo de vida.

Software Configuration Control Board (SCCB)

- Es quien tiene la autoridad de aceptar/rechazar un cambio en la configuración del sw.
- En proyectos pequeños puede residir en el Líder del Proyecto, y a medida que se incrementa la complejidad, puede ir incorporando nuevos integrantes.
- Puede haber diversos niveles de autorización de acuerdo a diferentes criterios



Change Management Process

- Flujo y herramientas definidas para identificar, analizar y aprobar cambios a una configuración. Difiere en cada organización, pero en general se puede dividir en las siguientes etapas:
 - **Identificar el cambio**
 - Incluye formalizar el pedido de cambio
 - **Evaluar el impacto**
 - Analizar el cambio y determinar el impacto que tendrá el sw, el equipo que trabaja en él y quienes lo usarán.
 - Evalúe los aspectos técnicos de cambios, sus posibles efectos secundarios y el impacto general en otras partes del sistema
 - **Decidir hacer o no el cambio**
 - Entendiendo el impacto en el negocio, o en el calendario o en el equipo.
 - **Planificar la implementación del cambio**
 - Incorporarlo dentro de las tareas a realizar

Mediciones relacionadas a Change Requests

- **Successful changes**
 - Cuenta la cantidad de cambios que han sido aprobados y desplegados en producción sin generar incidentes
- **Emergency changes**
 - Cuenta la cantidad de cambios que han sido aprobados por canales de urgencia o emergencia
- **Changes in backlog**
 - Mide la cantidad de cambios pendientes de ser analizados. Es un indicio de la eficiencia del proceso de aprobación de cambios y se puede utilizar para evaluar la velocidad del equipo

Status Accounting de la Configuración

Tiene la función de registrar y reportar información necesaria para administrar la configuración de manera efectiva. Para ello debe:

- Listar los IC's aprobados
- Mostrar el estado de los cambios que fueron aprobados
- Reportar la trazabilidad de todos los cambios efectuados al baseline

Una implementación exitosa de esta actividad debe poder contestar:

- Qué cambios se realizaron al sistema
- Cuando se realizaron dichos cambios

- Quien lo cambio
- Qué cambio

También responder acerca del proceso de cambios

- Quien aprobó el cambio
- Quien solicitó el cambio

Auditorías de la Configuración del sw

Examinación de un trabajo, producto o set de artefactos para evaluar aspectos técnicos, de seguridad, legales y de cumplimiento de normas con respecto a especificaciones, estándares, acuerdos contractuales u otros criterios

Las auditorías son llevadas a cabo de acuerdo a procesos definidos y pueden ser ejecutadas con diferentes niveles de formalidad, desde revisiones informales basadas en checklists hasta pruebas exhaustivas de la configuración que son planificadas con anticipación.

Auditoria funcional

- Evaluación del producto que se realiza para verificar, vía testing, inspección, demostración o análisis de resultados, que el producto ha cumplido los requerimientos especificados en la línea base de documento funcional.
- Se verifica que una configuración dada cumpla con alguna especificación de requerimientos previamente definida.

Auditoria fisica

- Evalúa que los elementos modificados de la configuración sean consistentes con las especificaciones técnicas de los cambios en la versión
- Evalúa los cambios realizados en el código y en los diferentes ambientes

Auditoria de proceso

- Evalúa que los cambios realizados al producto se hayan seguido el proceso definido para los mismos
- Confirmar que todos los cambios fueron solicitados, aprobados e implementados de acuerdo al proceso definido en la compañía.

Release Management

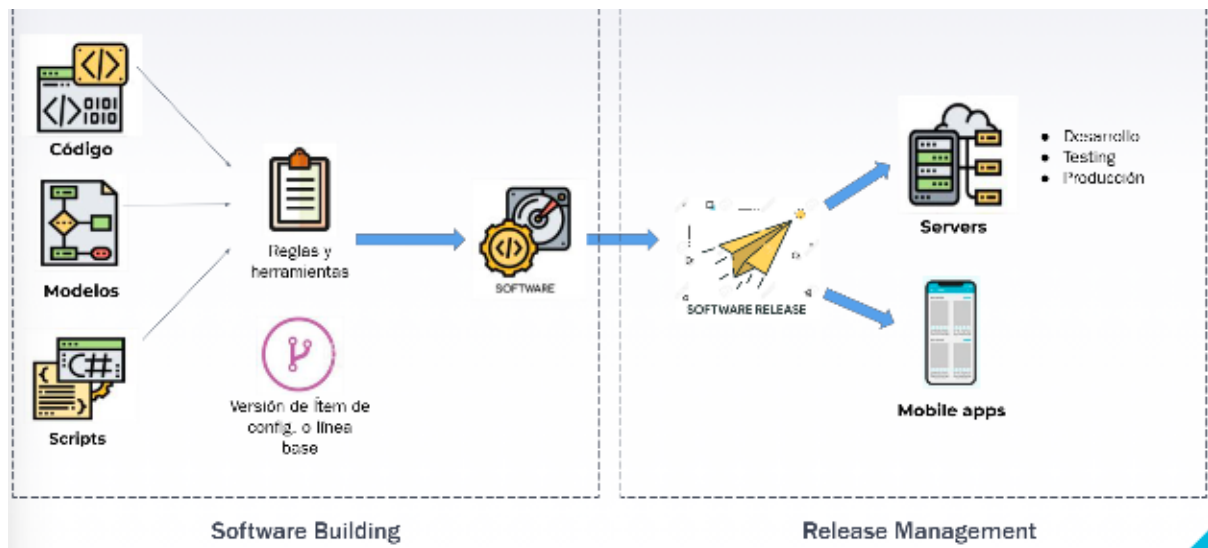
Consiste en asegurar la construcción exitosa del paquete de software, basada en IC's requeridos para la funcionalidad a entregar, para luego liberarlo en forma controlada a otros entornos, ya sea de pruebas, producción, usuario final, etc.

Se divide en 2 partes:

- Software building: construcción del paquete de software que será distribuido, utilizando las versiones correctas de los IC's y las herramientas apropiadas para construirlo.
 - Build: Proceso de convertir archivos de código fuente en artefactos de software independiente que se pueden ejecutar en un entorno
 - Local: El dev lo realiza localmente en su entorno de desarrollo, corre pruebas unitarias (UT)
 - Integración: su objetivo es generar el entorno completo para pruebas de integración

- Nightly: Su objetivo es ejecutar la construcción en forma diaria y generar reportes con información sobre estabilidad, tiempo de build, etc.
 - Release: Se disparan cuando bien un administrador decide crear una nueva versión a ser liberada, o por el mismo sistema de integración si se utiliza el modo de deployment continuo.
 - Release management: gestiona la identificación, ensamblado y entrega de los elementos de un producto software (ej. ejecutable, documentación, notas de lanzamiento o datos de configuración).
- Define también el proceso de planificación, calendarización y gestión del software construido a lo largo de las etapas de desarrollo, testing, despliegue y soporte productivo.
- Versión: Instancia de un sistema que es funcionalmente distinta en algún aspecto de otras instancias
 - Variante: Instancia de un sistema que es funcionalmente igual a otras instancias, pero difiere a niveles no funcionales (ej. mismo producto para versión linux/windows)
 - Release: En este contexto, hace referencia a la distribución del software fuera del entorno de desarrollo. En algunas herramientas, se utiliza el concepto de un "tag" como equivalente de release

Comprende también la administración, identificación y distribución de un producto software



Integración continua

Práctica de desarrollo del software mediante la cuál los desarrolladores combinan los cambios en el código en un repositorio central de forma periódica, tras lo cuál se ejecutan versiones y pruebas automáticas.

Se refiere en su mayoría a la fase de creación o integración del proceso de publicación del software y conlleva un componente de automatización y un componente cultural.

Es una práctica requerida para poder realizar entregas continuas (CD: Continuous Delivery)

Prácticas:

- Disponer de un repositorio de código único
- Automatizar el proceso de build mediante scripts o herramientas
- Hacer build testable automáticamente
- Todo commit debe construirse por una herramienta de integración. No por el dev
- El build debe ser rápido
- Se prueba en un clon de producción

- Cualquiera puede obtener fácilmente la última versión del ejecutable
- Todos deben poder ver qué pasa con el proceso de integración (transparencia)
- Automatizar el proceso de despliegue (deployment)

Responsabilidades del equipo:

- Hacer check-in del código frecuentemente
- No subir código roto
- No subir código no testeado
- No subir código cuando el build no pasa
- No irse a casa hasta que el build central compile.

Ventajas:

- Detección temprana y mejorada de errores, y métricas que le permiten abordar los errores a tiempo, a veces tras solo unos minutos de la incorporación.
- Progreso continuo para mejorar el feedback
- Mejor colaboración en equipo: todos los miembros del equipo pueden cambiar el código, integrar el sistema y determinar rápidamente los conflictos con otras partes del software
- Integración mejorada del sistema, lo que reduce las sorpresas al final del ciclo de vida de desarrollo del software
- Menor número de errores durante las pruebas del sistema.
- Sistemas actualizados constantemente en los que realizar las pruebas
- Fin del “en mi maquina funciona”

Semantic versioning

Propuesta de set de reglas y requerimientos que dictaminan como los números de versión son asignados e incrementados.

Dado un numero de version MAJOR.MINOR.PATCH, se incrementa:

- El numero de version MAJOR cuando se realizan cambios incompatibles en la API o SW
- El número de versión MENOR cuando se agrega funcionalidad de una manera compatible con versiones previas.
- El número de versión PATCH cuando se arreglan bugs de forma compatible con versiones anteriores.

Se pueden agregar etiquetas adicionales para pre-release o metadatos de build como extensiones del formato MAJOR.MINOR.PATCH

Consideraciones para release management

Una vez generada la versión o release debemos buscar el mecanismo más efectivo para hacer despliegue/deploy controlado a los distintos usuarios. A estos mecanismos se los denomina Deployment Strategies

Aspectos a evaluar para realizar un release:

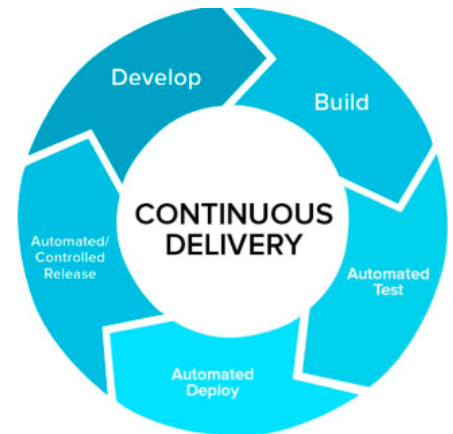
- Que usuarios deberán recibir los cambios (geográfico, premium, por segmento)
- En qué forma se deberá hacer el despliegue
- Entornos por los que deberá pasar (testing, staging, production, otros)
- Procesos de Roll Forward
- Procesos de Rollback
- Validación de despliegue correcto/incorrecto
- Riesgos del despliegue y cómo se minimizan
- Aprobación por el negocio y/o area de QA
- Quienes realizarán el deployment de la versión definida

Continuous Delivery

Conjunto de prácticas que permiten asegurar que el software puede ser desplegado en producción rápidamente y en cualquier momento, aplicando el concepto de pipelines.

Los artefactos resultantes del proceso de construcción incluyen código ejecutable y bibliotecas, que luego se puede combinar en una instalación paquete e implementado en un entorno para verificación o uso de producción.

Esto lo logra haciendo que cada cambio llegue a un entorno de staging o semi productivo, donde se corren exhaustivas pruebas del sistema. Luego se puede pasar a producción manualmente cuando alguien apruebe el cambio con solo apretar un botón



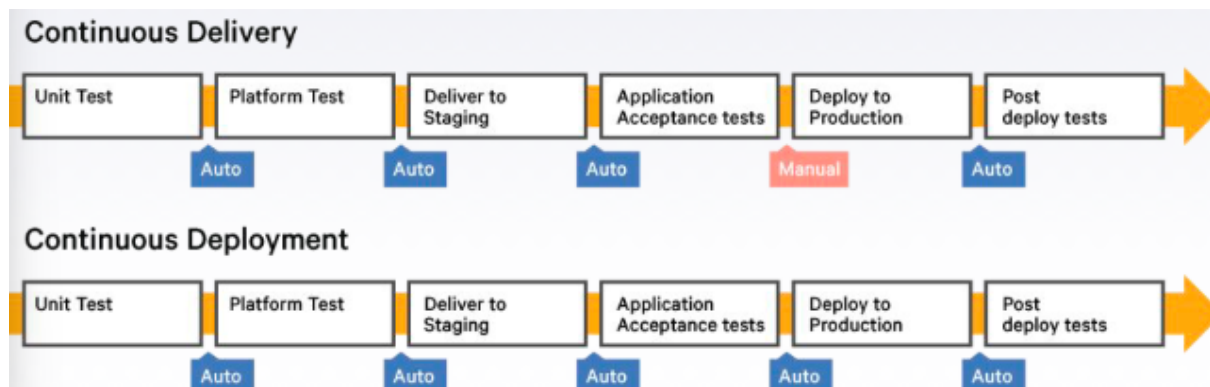
Pipelines

Para poder cumplir con el objetivo, los procesos de CD establecen un pipeline de acciones para poder construir e instalar el producto Software

Usualmente, estos pipelines suelen contener las siguientes etapas:

- **Testing y QA del software construido:** Ejecuta las pruebas automáticas
- **Change Management, Governance and Approvals:** Contiene las actividades de aprobación de los cambios a desplegar
- **Deployment strategies:** Define la estrategia con la que el software será desplegado en el ambiente
- **Verification:** Define las actividades para verificar que el software fue correctamente desplegado
- **Rollback:** Define las acciones para volver al estado anterior, en el caso de que falle la verificación

CD vs CD



El Continuous Deployment es el paso siguiente al Continuous Delivery, en donde también el despliegue a producción se realiza en forma automática por un proceso y no por personas.

Deployment Strategies

- Mecanismo para cambiar o actualizar una aplicación de software
- El objetivo es realizar el cambio sin generar un downtime, de forma tal que los usuarios finales no noten el cambio de aplicación
- Existen varias estrategias de despliegue

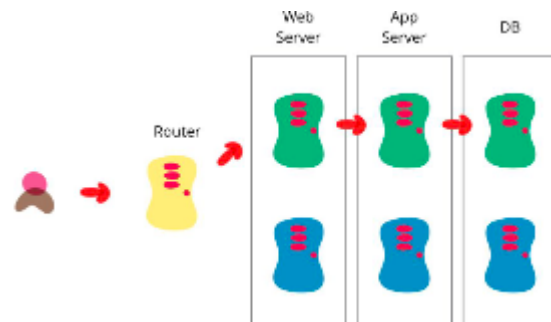
Basic Deployment

- **Concepto:** Todos los nodos de la aplicación son actualizados en el mismo momento con la nueva versión
- **Ventajas:** Muy simple de utilizar, no hay mecanismos específicos que debamos realizar para este deploy
- **Desventajas:** Riesgo alto de downtime



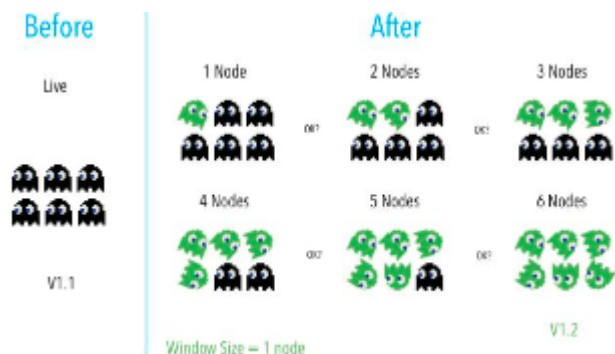
Blue/Green Deployment

- **Concepto:** Disponemos de dos ambientes lo más similares posible: uno green y otro blue. El despliegue se realiza sobre uno de los ambientes, mientras el otro contiene la versión anterior. Un balanceador selecciona a qué grupo de servers utilizar como productivo.
- **Ventajas:** Simple de implementar
- **Desventajas:** En organizaciones grandes, se torna una solución costosa.



Rolling Deployment

- **Concepto:** Los nodos de un ambiente son actualizados 1 a 1 (o en lotes) con la nueva versión del software.
- **Ventajas:** Rollout gradual con un incremento de tráfico controlado. Simple de hacer rollback
- **Desventajas:** App/BD necesitan mantener consistencias para trabajar con versiones distintas (+1/-1 en cada pod)



Canary Deployment

- **Concepto:** Consiste en desplegar el cambio en un set controlado de nodos, e ir chequeando el comportamiento. Es actualmente una de las formas más comunes de hacer deploy.
- **Ventajas:**
 - El despliegue es en pequeñas fases
 - En el caso de fallas, se quite los nodos que fueron incorporándose
 - Solo existe 1 ambiente de producción
- **Desventajas:** Es complejo de testear producción junto a herramientas de monitoreo complejas para entender que está desplegado.

Release Management Metrics

Métricas asociadas a los flujos de building y de releasing de software

Métricas relacionadas a deployment

Deployment frequency

Cada cuanto tiempo una organización despliega software a producción satisfactoriamente

- Medida: Cantidad de deployments por día
- ¿Para qué?: Indica cada cuanto tiempo agregamos valor a los clientes

Lead time for changes

El tiempo que le lleva a un commit llegar a producción

- Medida: Lead Time en días
- ¿Para qué?: Potencialmente puede indicar la productividad del proceso de desarrollo

Change failure rate

Porcentaje de despliegues que llevaron a una degradación del servicio

- Medida: Número de incidentes/Número de despliegues
- ¿Para qué?: Incrementar la satisfacción de los clientes al reducir el número de downtimes

Mean Time To Recovery (MTTR)

El tiempo que le demora a la organización recuperarse luego de un incidente

- Medida: Tiempo medido en horas
- ¿Para qué?: Incrementar la satisfacción de los clientes al reducir el número de downtimes.

SCM Tools

Las herramientas dan soporte a la gestión de configuración en diferentes niveles. Cada organización define qué herramientas utiliza en esta gestión.

Dentro de las herramientas, podemos mencionar:

- Herramientas que gestionan las versiones de los códigos fuente, archivos de configuración y artefactos relacionados.
- Herramientas que ayudan en la automatización del build y establecen mecanismos (via pipelines) para permitir el delivery continuo
- Repositorios que almacenan las historias de los builds
- Herramientas que gestionan los cambios funcionales y el control de cambios
- Herramientas que dan soporte al despliegue de aplicaciones
- Herramientas colaborativas para comunicar.

DevOps

Conjunto de prácticas destinadas a reducir el tiempo entre el cambio en un sistema y su pasaje a producción, garantizando la calidad y minimizando el esfuerzo.

Es también un cambio cultural, ya que en un modelo de DevOps, los equipos de desarrollo y operaciones ya no están “aislados”. A veces, los dos equipos se fusionan en uno solo, donde los ingenieros trabajan en todo el ciclo de vida de la aplicación, desde el desarrollo y las pruebas hasta la implementación y las operaciones, y desarrollan una variedad de habilidades no limitadas a una única función.

DevOps no es una metodología, es más bien un conjunto de prácticas, herramientas y una filosofía cultural que automatiza e integra los procesos entre los equipos de desarrollo de software y IT. El término se acuñó en una conferencia de infraestructura en Bélgica que quería atraer a desarrolladores.

CALMS

Filosofía de trabajo detrás de DevOps, donde:

- **Cultura:** ser dueños del cambio para mejorar la colaboración y comunicación
- **Automatización:** Eliminar el trabajo manual y repetitivo lleva a procesos repetibles y sistemas confiables, reduciendo el error humano.
- **Lean:** Remover la burocracia para tener ciclos más cortos y menos desperdicio
- **Métricas:** Medir todo, usar datos para refinar ciclos.
- **Sharing:** Compartir experiencias de éxito y fallas para que otros puedan aprender.

El problema a resolver

En organizaciones de las llamadas tradicionales

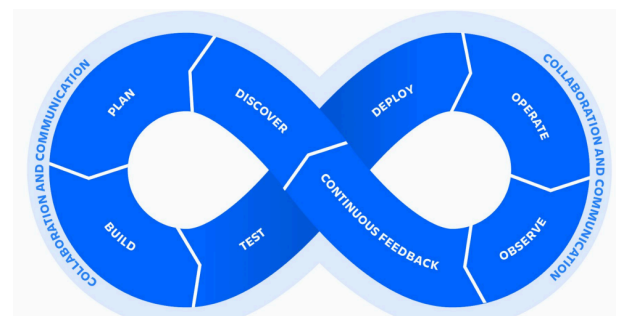
- El que desarrolla no escribe los requerimientos
- El que desarrolla no hace testing funcional
- El que desarrolla no pasa a producción
- El que desarrolla no está de guardia

Durante décadas los operadores de IT y sistemas vivieron en plena queja con desarrolladores que no consideraban:

- a. que los entornos pueden tener diferencias
- b. que la puesta en producción no es gratuita
- c. cuestiones como la escalabilidad, performance, disponibilidad, tolerancia a fallos, entre otros.
- d. Hacer software no es fire & forget, y esperaban que las operaciones maneje automáticamente cualquier problema, sin conocimiento del dominio.

Prácticas de DevOps en cada etapa

- Planificación del cambio:
 - Conversación y colaboración
- Coding & Building
 - Automate Build Pipelines
 - Infrastructure as a Code
- Testing
 - Automated Testing
 - Chaos Testing / Fault Injection
- Release & Deployment
 - CI/Integration / CDelivery
 - Operation & Monitoring
- Operation & Monitoring
 - Virtualization & Containers
 - Monitoring & Alerting tools



Software Testing

Conceptos generales

¿Qué es el testing?

Actividad para evaluar si los resultados obtenidos en la ejecución del software coinciden con los resultados esperados, con el propósito de determinar si el sistema está libre de defectos. Involucra la ejecución de un componente de software o de un sistema para evaluar una o varias propiedades de interés.

Las pruebas por sí solas encuentran fallas, NO defectos (a lo cuál solo pueden demostrar su presencia pero nunca su ausencia)

- Partamos de la suposición que el software a probar contiene defectos (lo cuál casi siempre es válido) y se trata de encontrar tantos como sea posible.
- Por el contrario, si nuestro objetivo es mostrar que el software no contiene errores, estaremos inconscientemente orientados a ese fin.
- El adoptar una u otra actitud tiene profundas implicancias en el éxito de la prueba.

Importancia

El testing es importante porque los bugs pueden ser costosos o incluso peligrosos. Los bugs de software pueden causar potencialmente problemas económicos o de pérdidas humanas.

- Cada vez existe software más crítico y más complejo
- El software está escrito por personas, y las personas cometen errores.
- Por lo tanto, el software puede cometer defectos que llevaran potencialmente a fallas.

Objetivo

Encontrar fallas en el producto.

Hay 2 formas:

1. Eficiente
 - a. Hacerlo lo más rápido posible
 - b. Hacerlo lo más barato posible
2. Eficaz
 - a. Encontrar la mayor cantidad de fallas
 - b. No detectar fallas que en realidad no son
 - c. Encontrar las más importantes

Causas de las fallas

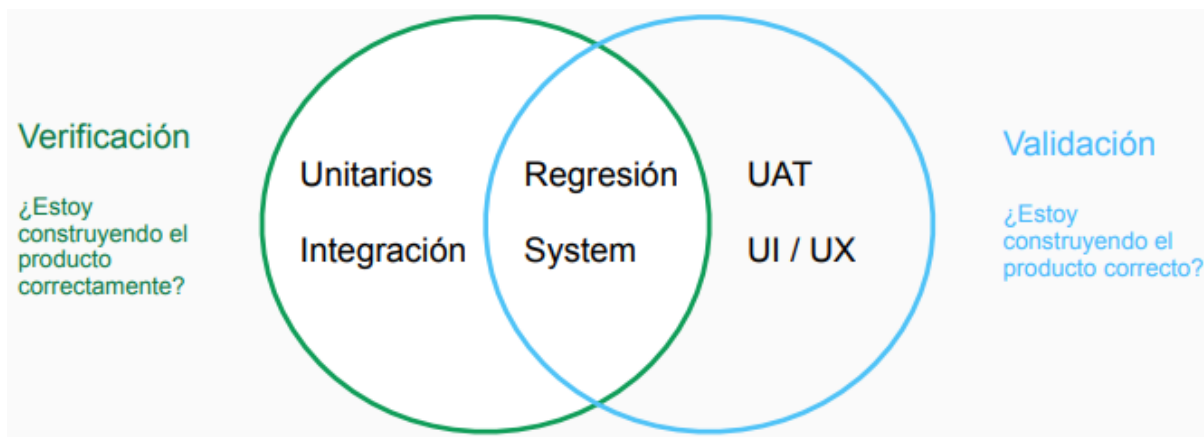
- Requerimientos poco claros; incorrectos o cambiantes
- Diseño complejo
- Desarrolladores sin experiencia
- Lógica complicada
- Falta de tiempo o de conocimiento del producto
- Testers sin experiencia
- Problemas del entorno o de sistemas.

Software de Calidad

- Aquel que cumpla con los requisitos
- El que al ser usado no tenga fallas
- Una vez definidos los requerimientos de calidad, tengo que tener en cuenta que:
 - La calidad no puede “inyectarse” al final
 - La calidad del producto depende de tareas realizadas durante el proceso

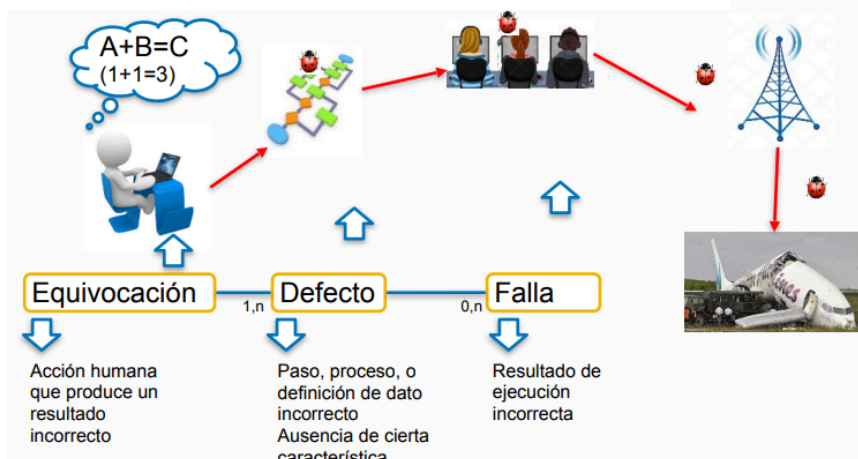
Aseguramiento de la calidad

- Patrón planificado y sistemático de todas las acciones necesarias para brindar una adecuada confianza que un producto o componente cumple con los requerimientos técnicos establecidos
- La prueba del software es una de las actividades involucradas en el aseguramiento de la calidad
- Detectar errores en forma temprana ahorra esfuerzos, tiempo y recursos



Proceso de testing

- Probar es ejecutar un componente con el objetivo de producir fallas
- Una prueba es exitosa si encuentra fallas



Incidentes

- Toda ocurrencia de un evento que ocurre durante la ejecución de una prueba de software que requiere investigación
- Un incidente puede deberse a:
 - Un defecto en el software
 - Un defecto en los casos de prueba
 - Un defecto en el ambiente
- No todo incidente es una falla, por ejemplo:
 - Defectos en los casos
 - Equivocaciones al ejecutar las pruebas
 - Interpretaciones erróneas
 - Dudas

Depuración

- Eliminar un defecto que posee el software
- No es una tarea de prueba aunque es una consecuencia de ella.
- La prueba detecta el fallo (efecto) de un fallo (causa)
- La depuración puede ser fuente de introducción de nuevos defectos

Proceso

- Detectar
 - Dada la falla debemos hallar el defecto (dado el efecto debemos encontrar la causa)
- Depurar
 - Encontrado el defecto, debemos eliminarlo
 - Debemos encontrar la razón del defecto
 - Debemos encontrar una solución
 - Debemos aplicarla
- Volver a probar
 - Asegurar que sacamos el defecto
 - Asegurar que no hemos introducido otros (regresión)
- Aprender para el futuro
 - Lecciones aprendidas

Economía del testing

¿Hasta cuándo tengo que probar?

- Se puede invertir mucho esfuerzo en probar
- Probar es el proceso de establecer confianza en que un programa hace lo que se supone que tiene que hacer
- Ya que nunca voy a poder demostrar que un programa es correcto, continuar probando es una decisión económica

¿Cuándo parar?

- Nuestro problema
 - Imposibilidad en la práctica de estimar la intensidad del testing para alcanzar determinada densidad de defectos
- Estrategias
 - Pasa exitosamente el conjunto de pruebas diseñado
 - “Good enough”: Cierta cantidad de fallas no críticas es aceptable (umbral de fallas no críticas por unidad de testing)
 - Cantidad de fallas detectadas es similar a cantidad de fallas estimadas

Abaratamiento del testing

- Hacer pruebas es caro y trabajoso
- La forma de abaratarlas y acelerarlas - sin degradar su utilidad - es:
 - Diseñando el software para ser testeado
- Algunas herramientas son:
 - Diseño modular
 - Ocultamiento de información
 - Uso de puntos de control
 - Programación no egoísta

Enfoques de prueba

Caja negra

- Prueba funcional, producida por los datos, o producida por la entrada/salida
- Prueba lo que el software debería hacer
 - Se basa en la definición del módulo a probar (definición necesaria para construir el módulo)
 - Nos desentendemos completamente del comportamiento y estructura interna del componente
- ¿Qué hacemos?
 - Seleccionamos subconjuntos de los datos de entrada posibles, esperando que cubran un conjunto extenso de otros casos de prueba posibles
 - Podemos suponer que la prueba de un valor representativo de cada clase es equivalente a la prueba de cualquier otro valor
 - Llamamos a cada subconjunto “Clase de equivalencia”
 - Estas pruebas son llamadas “Pruebas por partición de equivalencia” o “Pruebas basadas en subdominios”

Condiciones de prueba

Son descripciones de situaciones que quieren probarse ante las que es el sistema debe responder

Casos de prueba

Son lotes de datos necesarios para que se de una determinada condición de prueba

Criterio de selección de casos

- Condición para seleccionar un conjunto de casos de prueba
- De todas las combinaciones posibles solo seleccionaremos algunas:
 - La menor cantidad de aquellas que tengan mayor probabilidad de encontrar un defecto no encontrado por otra prueba

Enfoque de caja negra

- Variaciones de eventos
- Clases de equivalencia
 - De entrada
 - De salida
- Condiciones de borde
- Ingreso valores de otro tipo
- Integridad del modelo de datos
 - De dominio
 - De entidad
 - De relación

Partición de casos

- Todos los posibles casos de prueba los dividimos en clases
- Todos los casos de una clase son equivalentes entre sí
 - Detectan los mismos defectos
 - Con los ejemplos de cada clase cubrimos todas las pruebas
- El éxito está en la selección de la partición

Enfoque caja blanca

- Prueba estructural del software
- Es un método de software testing en el cuál la estructura interna, el diseño y la implementación del software es conocido por el tester
- El tester elige qué valores de entrada utilizar para ejecutar ciertos caminos en el código, conociendo la potencial salida de ese camino
- En este tipo de pruebas, es esencial que el tester conozca el lenguaje de programación con el que el software fue construido, debido a que se basa en cómo está estructurado el componente internamente y su definición
- Se lo conoce como caja blanca porque es desde el punto de vista del tester, se puede ver como el software está construido
- Se utiliza también para incrementar el grado de cobertura de la lógica interna del componente.

Grados de cobertura

- Cobertura de sentencias: Prueba cada instrucción
- Cobertura de decisiones: Prueba cada salida de un if o while
- Cobertura de condiciones: Prueba cada expresión lógica de los if, while, etc.
- Prueba del camino básico: Prueba todos los caminos independientes

Complejidad ciclomática

- Métrica del software que proporciona una medición cuantitativa de la complejidad lógica de un programa
 - Cantidad de caminos independientes
 - Camino independiente: Agrega un nuevo conjunto de sentencias de procesamiento o una nueva condición.
- Formas de calcularla
 - Número de regiones en el grafo 🙅
 - $V(g) = \#Aristas - \#Nodos + 2$
 - $\#Condicionales (if, else if, while, for, case (cada rama del case suma)) + 1$

Generación de condiciones y casos

- Ninguna técnica es completa
- Las técnicas atacan distintos problemas
- Lo mejor es combinar varias de estas técnicas para complementar las ventajas de cada una
- Sin especificaciones de requerimientos todo es muchísimo más difícil

Tipos de prueba

Prueba unitaria

- Se realiza sobre una unidad de código claramente definida
- Generalmente lo realiza el área que construyó el modelo
- Se basa en el diseño detallado
- Comienza una vez codificado, compilado y revisado el módulo
- Los módulos altamente cohesivos son los más sencillos de probar

Prueba de integración

- Orientada a verificar que las partes de un sistema que funciona bien aisladamente, también lo haga en su conjunto
- Tipos
 - No incrementales
 - Big Bang
 - Incrementales
 - BottomUp
 - TopDown
 - Sandwich
- Los puntos clave son:
 - Conectar de a poco las partes más complejas
 - Minimizar la necesidad de programas auxiliares

Prueba de aceptación de usuario

- Prueba realizada por los usuarios para verificar que el sistema se ajusta a sus requerimientos

- Las condiciones de prueba están basadas en el documento de requerimientos
- Es una prueba de “caja negra”

Prueba de stress

- Orientada a someter al sistema **excediendo** los límites de su capacidad de procesamiento y almacenamiento
- Teniendo en cuenta situaciones no previstas originalmente

Pen Testing / Ethical Hacking

- Es la práctica de testear un software, red o aplicación web con el fin de encontrar vulnerabilidades de seguridad que un hacker pudiera explotar
- El test en sí puede automatizarse utilizando aplicaciones de software o realizado en forma manual. De cualquier manera, el proceso involucra obtener información acerca del software o servicio a probar, identificar potenciales puntos de entrada y reportar lo encontrado

Prueba de volumen

- Orientada a verificar que el sistema soporta los volúmenes máximos **definidos** en la cuantificación de requerimientos
 - Capacidad de almacenamiento
 - Capacidad de procesamiento

Prueba de regresión

- Orientada a verificar que, luego de introducido un cambio en el código, la funcionalidad original no ha sido alterada
 - Hay que probar lo viejo
 - Reuso de condiciones y casos

Prueba alfa y beta

- Se entrega una primera versión al usuario que se considera estar lista para ser probada por ellos.
 - Normalmente plagada de defectos
 - Una forma económica de identificarlos (ya que el trabajo lo hace otro)
 - En muchos casos no puede hacerse
 - Alternativa válida: ejecutar en “paralelo”
 - Alfa: la hace el usuario en mis instalaciones
 - Beta: la hace el usuario en sus instalaciones

Conclusiones

- Las pruebas no mejora el software, solo muestran cuántas fallas se han producido debido a distintos tipos de defectos
- El buen diseño y construcción no solo benefician a las pruebas, sino también a la corrección de los componentes y su mantenimiento.

- El no probar no elimina los errores, ni acorta tiempos, ni abarata el proyecto
- Lo más barato para encontrar y eliminar defectos es no introducirlos
- Hay herramientas que ayudan a automatizar las pruebas, pero requieren esfuerzo para crear y mantener y ayudan aunque no son suficientes.

Métricas

Conceptos introductorios

- Una medida proporciona una indicación cuantitativa de la extensión, cantidad o tamaño de algunos atributos de un proceso o producto. Ej: un programa tiene 10.000 líneas de código
- Una métrica es una medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo dado. Ej: la productividad de un proyecto es de x líneas de código por persona por mes
- Un indicador es una métrica o combinación de métricas que proporcionan una visión más profunda de un proceso, un proyecto de software o de un producto en sí

Indicadores (KPI)

- Objetivo
- Interpretación/Descripción
- Metodo de medicion
 - Fuente/s de datos
 - Fórmula
 - Visualización gráfica
- Frecuencia de cálculo
- Valores esperados o valores normales, rangos
- Representación (gráfica - numérico)

Pasos para implementar un programa de métricas

1. Identificar los objetivos (**GQM**)
2. Hallar las preguntas que me ayuden a alcanzar el objetivo (**GQM**)
3. Definir las métricas (**GQM**)
4. Recolectar datos históricos
5. Definir el proceso de recolección de datos para las métricas
6. Recolectar, validar y analizar los datos
7. Utilizar las métricas en la toma de decisiones

GQM (Goal - Question - Metric)

1. Conceptual (Goal): aquello que la organización intenta alcanzar (SMART)
2. Operacional (Question): son aquellas preguntas cuyas respuestas permiten definir el cumplimiento de las metas
3. Cuantitativo (Metrics): las mediciones necesarias para ayudar a responder a las preguntas y confirmar si las mejoras del proceso cumplieron su objetivo

Clasificación de métricas

- Software: cualquier artefacto o entregable que resulta del proceso de desarrollo de software
 - Cantidad de líneas de código
 - Funcionalidad (Use case points)
 - Complejidad ciclomática
 - Cohesión
 - Acoplamiento
 - Calidad de producto
 - Cantidad de fallas en un producto
 - Confiabilidad
 - MTBF: Mean Time Between Failures
 - MTTR: Mean Time to Recovery
- Proceso: colección de actividades relacionadas con el desarrollo y mantenimiento de software
 - Duración promedio de proyectos
 - Cantidad de proyectos segregados
 - Esfuerzo promedio segregado
 - Defectos introducidos en una fase del ciclo de vida
 - Defectos detectados en una fase del ciclo de vida
 - % de tiempo/esfuerzo/costo dedicado a una fase del ciclo de vida
 - % promedio de desvío en proyectos (Costo o duración)
- Recursos: Cualquier entidad requerida para realizar un proceso o actividad
 - Cantidad de fallas detectadas por tester
 - Cantidad de LDC producidas por un desarrollador
 - Esfuerzo dedicado a codificar por un desarrollador
 - Edad promedio del equipo
 - Costo promedio de cada rol
 - Años de experiencia promedio de cada rol

Conclusiones

- Las métricas representan datos objetivos
 - Se dejan de lado las sensaciones estomacales/subjetivas
 - Se pueden comparar
- Recomendaciones
 - Medir pocos indicadores simples
 - Se debe hacer una selección de métricas a implementar basada en objetivos
 - Hay muchas métricas a utilizar, no comenzar midiendo todas a la vez
 - Detectar aquellos más relevantes
 - No buscar resultados en el corto plazo
 - Sostener el esfuerzo de recolección en el tiempo
 - En lo posible automatizar la captura de información para la obtención de métricas
 - No medir personas ni evaluar de acuerdo a las métricas
 - Utilizar las métricas para la toma de decisiones